DAY 28

# DATABRICKS vs SNOWFLAKE

## The Interview Trap

**THE 90-SECOND ANSWER**

Do not answer "Databricks is a data lake and Snowflake is a warehouse." That is now an incomplete mental model. Both platforms span data engineering, analytics, governance, streaming, ML, and AI. The better answer starts with workload and architecture, then explains where each platform is naturally strongest.

### The modern mental model

| Databricks | Snowflake |
|---|---|
| Lakehouse-first Data + AI platform | Cloud data platform with warehouse-first roots |
| **Architectural roots** | **Architectural roots** |
| Apache Spark + open lakehouse + data engineering / ML | Cloud data warehousing + SQL analytics + managed experience |

### What both can do today

| | | |
|---|---|---|
| **Data Engineering** | **Analytics** | **Streaming** |
| Batch + pipelines | SQL + BI | Near-real-time |
| **Governance** | **ML** | **AI** |
| Catalog + policies | Model workflows | GenAI capabilities |

**ONE-LINE INTERVIEW TAKEAWAY**

"I would not choose by brand. I would first evaluate where the data lives, the dominant compute model, SQL vs Spark needs, batch vs streaming, AI/ML requirements, openness, governance, team skills, concurrency, and economics."

Verified against current official Databricks and Snowflake documentation. Product capabilities evolve quickly; the comparison below focuses on durable architectural differences rather than marketing checklists.

DAY 28 | 01

# Databricks: Lakehouse-first architecture

Understand where data lives, what executes the work, and why Databricks feels natural for Spark-centric engineering and AI workloads.

## 1. Storage and table layer

- Databricks lakehouse decouples compute from storage and commonly uses cloud object storage as the durable data layer.
- Delta Lake is the default table format on Databricks and adds ACID transactions, scalable metadata, schema enforcement, and deep Spark / Structured Streaming integration.
- Current Databricks reference architectures also support Apache Iceberg tables, so the platform is not limited to one open table format.

## 2. Simplified execution path

| Cloud Object Storage | Table Layer | Compute Engines | Workloads | Consumers |
|---|---|---|---|---|
| S3 / ADLS / GCS | Delta / Iceberg | Spark / Photon | ETL / SQL / Streaming | BI / ML / AI |

## 3. What makes Databricks different?

- Spark is foundational. Databricks is built on Apache Spark, and its platform is designed around large-scale distributed processing.
- Photon accelerates supported SQL/DataFrame execution while keeping the broader Spark programming model intact.
- Batch and streaming share the same lakehouse foundation; Structured Streaming and Lakeflow are first-class data-engineering patterns.
- Databricks SQL is a cloud data warehouse built on lakehouse architecture, so “Databricks cannot do warehousing” is outdated.
- Unity Catalog provides unified governance for data and AI assets, including access control, discovery, auditing, and lineage.

> **WHEN DATABRICKS NATURALLY FITS**
>
> Organizations with Spark-heavy engineering, large-scale ETL, streaming, open lakehouse requirements, data science, ML/AI, or a desire to run multiple data/AI workloads over the same governed storage layer.

## Important nuance: “open” does not mean “no platform services”

The data layer can use open formats and cloud object storage, but the Databricks platform still supplies managed compute, governance, orchestration, optimization, serving, and AI services around that data. The architectural benefit is avoiding a hard separation between an analytics warehouse and a separate engineering/ML lake.

DAY 28 | 02

# Snowflake: warehouse-first roots, broader platform today

Snowflake still has a distinctive three-layer architecture, but its current platform extends far beyond classic SQL warehousing.

## 1. Core architecture

| | |
|---|---|
| **Cloud Services** | Authentication, metadata, query optimization, access control, catalog/governance coordination. |
| **Compute** | Virtual warehouses: independent compute clusters that execute SQL and Snowpark workloads. |
| **Storage** | Durable storage managed by Snowflake for native tables; Iceberg can also use Snowflake-managed or external cloud storage. |

## 2. Why virtual warehouses matter

- Each virtual warehouse is an independent compute cluster and does not share compute resources with other virtual warehouses.
- This makes workload isolation intuitive: BI, ETL, data science, and other workloads can use separate compute and scale independently.
- Warehouse sizing, concurrency, and auto-suspend/resume become central cost/performance levers.

## 3. Snowflake is no longer “SQL only”

- Snowpark lets developers execute Python, Java, and Scala workloads using Snowflake compute.
- Snowpark Connect for Spark allows Apache Spark workloads to run on Snowflake virtual warehouses.
- Dynamic Tables provide declarative, refresh-driven transformations and can use incremental refresh where supported.
- Snowpipe Streaming provides low-latency streaming ingestion into Snowflake.
- Cortex and Snowflake ML expand the platform into generative AI and machine-learning workloads.
- Apache Iceberg tables can store files in Snowflake-managed storage or in external cloud storage that you manage.

> **WHEN SNOWFLAKE NATURALLY FITS**
>
> Organizations centered on SQL analytics and BI, strong workload isolation through independent warehouses, a highly managed platform experience, and teams that want to extend into engineering/AI without operating a Spark-first platform.

## 4. Governance

Snowflake Horizon Catalog spans cataloging, discovery, lineage, sensitive-data protection, data quality, access controls, and AI governance across Snowflake and supported external data estates.

DAY 28 | 03

# Architecture comparison: what actually differs

A useful comparison separates historical roots from current capabilities. “Both can do it” is not the same as “both do it the same way.”

| Dimension | Databricks | Snowflake |
|---|---|---|
| **Primary architectural idea** | Lakehouse: open data layer + multiple compute/workload types over governed data | Integrated cloud data platform with three-layer architecture and virtual warehouses |
| **Storage orientation** | Cloud object storage is central; Delta is default, Iceberg supported | Native Snowflake storage plus Iceberg options including external customer-managed storage |
| **Compute model** | Spark / Photon / SQL warehouses / serverless compute depending on workload | Independent virtual warehouses; additional serverless services depending on feature |
| **Data engineering identity** | Spark-native, Lakeflow, Structured Streaming, notebooks/jobs | SQL-centric transformations plus Snowpark, Dynamic Tables, Snowpipe Streaming, Spark options |
| **SQL / BI** | Databricks SQL is a cloud data warehouse on lakehouse data | Mature warehouse-first SQL analytics model with independent warehouse scaling |
| **Streaming** | Structured Streaming + Lakeflow; same table layer for batch/streaming | Snowpipe Streaming ingestion + streams/tasks/dynamic-table patterns |
| **ML / AI** | Integrated data science, MLflow, model serving, AI/agent capabilities | Snowflake ML + Cortex AI capabilities integrated with Snowflake data |
| **Governance** | Unity Catalog | Horizon Catalog |
| **Openness** | Strong emphasis on open formats, Spark ecosystem, direct data access | Increasing interoperability through Iceberg/Open Catalog while preserving Snowflake platform experience |
| **Best decision lens** | Engineering/AI + open lakehouse + Spark-oriented workloads | Managed SQL analytics + isolated warehouse compute + Snowflake-centric ecosystem |

Note: These are architectural tendencies, not hard feature boundaries. Both vendors continue to add overlapping capabilities.

DAY 28 | 04

# How to choose: use-case-first decision framework

A senior engineer should translate requirements into architectural consequences before choosing a platform.

| | |
|---|---|
| **1. Where will the data live?**<br>Cloud object storage as a shared lakehouse layer, Snowflake-managed storage, external Iceberg, or a mix? | **2. What dominates the workload?**<br>SQL/BI, Spark-heavy ETL, Python engineering, streaming, ML/AI, or a balanced mix? |
| **3. Batch or streaming?**<br>What latency is required: daily/hourly batch, near-real-time, or continuous processing? | **4. How important are open formats?**<br>Do external engines need direct interoperability with the same table data? |
| **5. What does the team know?**<br>SQL analysts, Spark/Python data engineers, ML engineers, platform engineers? | **6. How should workloads isolate and scale?**<br>Independent warehouse-style isolation or shared/open lakehouse data with different compute services? |
| **7. Governance requirements?**<br>Catalog, lineage, fine-grained access, data quality, audit, cross-platform governance? | **8. Economics?**<br>Concurrency, idle time, autoscaling, storage duplication, data movement, operational overhead, and predictable utilization. |

## Scenario shortcuts

**Spark-heavy enterprise platform:** Databricks often has the architectural advantage when Spark, large ETL, streaming, data science, and AI are core requirements.

**SQL/BI-heavy analytics estate:** Snowflake often has a natural advantage when SQL analytics, BI concurrency, isolated compute warehouses, and a managed warehouse experience dominate.

**Open table + multi-engine strategy:** Evaluate Delta/Iceberg interoperability, external engines, catalog strategy, and whether direct object-store access is a requirement.

**Mixed modern data platform:** Do a workload-by-workload TCO and operating-model comparison. It may be rational to use one platform broadly—or both selectively.

> **THE SENIOR-LEVEL PRINCIPLE**
>
> Feature parity is not architecture parity. Compare where data is stored, how compute is provisioned and isolated, how transformation/streaming executes, what governance boundary you want, and how your team will operate the platform.

DAY 28 | 05

# Interview Q&A + 90-second reel script

Use these answers to avoid the common “warehouse vs lake” oversimplification.

**Q1. Is Databricks a data lake and Snowflake a data warehouse?**

Not anymore as a complete comparison. Databricks provides full cloud data warehousing through Databricks SQL, while Snowflake has expanded into engineering, streaming, Iceberg, ML, and AI.

**Q2. What is the biggest architectural difference?**

Databricks is lakehouse-first and built around open storage/table formats plus Spark/Photon compute. Snowflake retains a distinct storage/compute/cloud-services architecture centered on independent virtual warehouses.

**Q3. Does Snowflake keep all data inside Snowflake storage?**

No. Apache Iceberg tables can use either Snowflake-managed storage or an external volume in customer-managed cloud storage.

**Q4. Can Snowflake run Spark?**

Yes. Snowflake documents Snowpark Connect for Spark, which runs Spark workloads on Snowflake virtual warehouses. That does not make its architecture identical to a Spark-native lakehouse.

**Q5. Which is better for streaming?**

Avoid a universal answer. Databricks has a strong Spark Structured Streaming/Lakeflow model; Snowflake has Snowpipe Streaming and related pipeline features. Evaluate latency, transformations, stateful processing needs, ecosystem, and operating model.

**Q6. Which one is cheaper?**

There is no universal winner. Cost depends on workload shape, concurrency, idle time, compute sizing, data movement/storage, serverless usage, optimization, and operational overhead. Benchmark representative workloads.

**Q7. What should I say if the interviewer asks “Databricks or Snowflake?”**

Start with “It depends on the workload and architecture.” Then ask about data location, SQL vs Spark, batch vs streaming, ML/AI, open formats, governance, team skills, concurrency, and economics before giving a recommendation.

## 90-second reel script: condensed version

> **SCRIPT**
>
> “Databricks better hai ya Snowflake? Interview mein directly kisi ek ka naam mat lena—correct answer hai: it depends on architecture and workload. Pehle outdated comparison hatao: Databricks sirf data lake nahi hai, aur Snowflake sirf data warehouse nahi hai. Databricks lakehouse-first platform hai—cloud object storage, Delta/Iceberg, Spark, Photon, SQL, streaming, ML aur AI ko ek governed platform par bring karta hai. Snowflake ka architectural DNA three layers mein hai: storage, compute aur cloud services. Uske virtual warehouses independent compute clusters hote hain, isliye BI, ETL aur other workloads separately scale aur isolate kiye ja sakte hain. Lekin Snowflake bhi evolve ho chuka hai—Snowpark, Spark options, Dynamic Tables, Snowpipe Streaming, Iceberg aur Cortex/ML available hain. Governance mein Databricks ke paas Unity Catalog hai aur Snowflake ke paas Horizon Catalog. Toh senior answer feature list nahi hai. Pehle poochho: data kahan rahega, SQL ya Spark, batch ya streaming, ML/AI kitna important hai, open formats chahiye, team skills kya hain, governance, concurrency aur cost kya hai. Choose architecture, not logo.”

DAY 28 | 06

# Official references

Primary sources used to verify the technical statements in these notes.


**1. Databricks — What is a data lakehouse?**

https://docs.databricks.com/aws/en/lakehouse/

Lakehouse definition, Spark foundation, Delta Lake, Unity Catalog, compute/storage separation.

**2. Databricks — Data warehousing on Databricks**

https://docs.databricks.com/aws/en/sql

Databricks SQL as a cloud data warehouse built on lakehouse architecture.

**3. Databricks — What is Delta Lake?**

https://docs.databricks.com/aws/en/delta

Delta Lake default format, ACID transactions, scalable metadata, Structured Streaming integration.

**4. Databricks — Reference architectures**

https://docs.databricks.com/gcp/en/lakehouse-architecture/reference

Cloud object storage, Delta/Iceberg, Spark/Photon, batch/streaming, MLflow and platform architecture.

**5. Snowflake — Key concepts and architecture**

https://docs.snowflake.com/en/user-guide/intro-key-concepts

Storage / compute / cloud services architecture, independent virtual warehouses, Snowpark and Spark.

**6. Snowflake — Apache Iceberg table storage**

https://docs.snowflake.com/en/user-guide/tables-iceberg-storage

Snowflake-managed and external-volume Iceberg storage options.

**7. Snowflake — Horizon Catalog**

https://docs.snowflake.com/en/user-guide/snowflake-horizon

Governance, discovery, lineage, data quality, data protection and AI governance.

**8. Snowflake — Dynamic Tables**

https://docs.snowflake.com/en/user-guide/dynamic-tables/create

Declarative materialized query results with target lag and incremental/full refresh modes.

**9. Snowflake — Snowpipe Streaming**

https://docs.snowflake.com/en/user-guide/snowpipe-streaming/snowpipe-streaming-high-performance-rest-api

Streaming ingestion interfaces and semantics.


> **FINAL REMINDER**
>
> Databricks and Snowflake are converging in capabilities, but their architectural roots still influence how teams design, operate, scale, govern, and pay for workloads. That is the comparison worth learning.